



中国科学技术大学
University of Science and Technology of China

Life Lab: AI Agent 社会实验平台 设计文档

赵俊宇 刘福康 岳雨婷

(中国科学技术大学 信息科学技术学院)

摘 要

本文档描述 AI Agent 社会实验平台 Life Lab 的系统设计。平台采取“快慢结合”、多链路并行的设计思想：关键词匹配能快速给出初步判断，但不够准确；LLM 深度评估需要数秒，但更可靠。两条独立路径互相保险，对抗 LLM 可能产生的错误。在此哲学之上，平台实现了三项核心能力：人格驱动的社会涌现（MBTI 人格直接决定 Agent 怎么移动、怎么社交、怎么产生情绪）、可信赖的多 Agent 协作（程序直接验证每一步是否真正执行）和可解释的行为评测（六个维度打分 + 信息论量化行为一致性 + 逐维度诊断证据链）。本文档重点阐述 M9-M12 四个进阶模块的设计决策，揭示从基础模块到进阶能力的生长脉络。

核心亮点

- ”多链路设计”：关键词匹配 1ms 出结果，LLM 深度评估数秒但更可靠，双路径互保
- 人格直接决定行为：MBTI 等可直接驱动对话/移动/情绪三维度，从”说什么”到”怎么走”全链路一致
- 程序直接验证交付：算式直接算一遍、工具记录直接查一遍，不依赖 LLM 判断
- 用信息论量化行为 + 跨任务鲁棒性 + 逐维度诊断，确保行为评测从定性走向定量
- 20 页面 + Phaser 3 + 图形化管线编辑器，保障系统完整度与工程成熟度

关键词：AI Agent；系统架构；人格建模；多智能体协作；行为指纹；管道编排

目录

1	引言	5
1.1	设计目标	5
1.2	设计原则	5
1.3	竞品对比与设计定位	5
2	系统总体架构	6
3	基础模块设计 (M1–M8)	6
3.1	人格数据模型 (M1)	7
3.2	世界引擎与关系双轨评估 (M2/M3)	7
3.3	叙事、竞技与干预 (M4–M8)	8
4	团队协作引擎 (M9)	9
4.1	设计决策一：文件系统与消息通信	9
4.2	设计决策二：LLM 分解 + 规则兜底	9
4.3	设计决策三：团队诊断与容错	9
5	评测基准 (M10)	9
5.1	设计决策一：鲁棒性为什么是跨任务指标	10
5.2	设计决策二：行为指纹为什么用信息熵	10
5.3	设计决策二-b：决策风格如何量化分类	11
5.4	设计决策三：诊断评分的设计动机	11
6	游戏化场景 (M11)	11
6.1	设计决策一：MBTI 到行为的映射	11
6.2	设计决策二：情绪引擎的多重触发	12
6.3	设计决策三：目标预订防碰撞协议	13
7	管道编排引擎 (M12)	13
7.1	设计决策一：Loop/Branch 的设计动机	13
7.2	设计决策二：确定性硬校验的设计动机	13
7.3	设计决策三：自动修订机制	14

8 技术亮点总结	14
9 前端交互层	15
10 数据库设计	16
11 总结	16

1 引言

1.1 设计目标

1. **人格真实性**: Agent 行为应与其人格参数保持一致, 不同人格在相同场景下表现出可辨识的行为差异。
2. **可扩展性**: 引擎层通过 Mixin 模式组合能力, 新功能添加不影响既有系统。
3. **实时性**: 通过 SSE 实现毫秒级事件推送, 前端实时渲染 Agent 思维流和行为。
4. **鲁棒性**: LLM 调用具备超时、重试和降级机制, 关键路径均有规则兜底方案。

1.2 设计原则

- **引擎与 API 分离**: 核心逻辑封装在引擎层 (engines), API 层仅负责路由和参数校验。
- **LLM 优先 + 规则兜底**: 关键决策优先使用 LLM, 不可用时降级为规则引擎。
- **多链路设计**: 关键词匹配能在 1ms 内给出初步判断 (每 tick, $O(n)$), 但不够准确; LLM 深度评估需要数秒, 但更可靠。两条路径**独立运行、互为保险**, 快的保证实时性, 慢的保证准确性。形式化地, 最终判定为 $D = D_{fast} \oplus D_{slow}$, 任一路径异常即触发告警。这一模式在关系评估、任务分解、交付验收等多个模块中反复出现, 是平台的核心工程哲学。

1.3 竞品对比与设计定位

	人格建模	行为评测	游戏化场景	交付验证
AutoGen / CrewAI	✓ (基础)	—	—	—
LangGraph	—	—	✓ (静态可视化)	—
AgentBench	—	✓ (黑箱评分)	—	—
Life Lab	✓	✓ (可解释)	✓ (实时交互)	✓

现有 Agent 框架各有侧重, 但均缺乏”人格-行为-评测”的完整链路。Life Lab 的独特定位是: 不仅是 Agent 编排工具, 更是 Agent 社会实验平台。

2 系统总体架构

系统采用四层架构：

1. **表现层**：React + TypeScript + TailwindCSS + Phaser 3，通过 React Query 管理服务端状态，SSE 接收实时事件。
2. **接口层**：FastAPI 路由层，提供 RESTful API 和 SSE 端点。
3. **引擎层**：persona/、agent_factory/、world/、narrative/、arena/、team/、bench/、worker/ 八大引擎。
4. **数据层**：Pydantic 模型 + SQLAlchemy 异步 ORM + SQLite/aiosqlite。



【系统总体架构图待插入】

Figure 1: Life Lab 四层架构设计

3 基础模块设计 (M1-M8)

基础模块覆盖 Agent 从”诞生”到”沉淀”的完整生命周期，为进阶能力提供不可或缺的人格基底和行为数据。

3.1 人格数据模型 (M1)

组件	类型	说明
MBTI	string	16 种人格类型 + T/A 变体
BigFive	struct	五维浮点参数 (0-1): 开放性、尽责性、外向性、宜人性、神经质
DecisionStyle	struct	四维决策风格: 信息处理、风险偏好、社交倾向、压力应对
Values	list[string]	核心价值观列表
Narrative	string	200-400 字人格画像
Background	struct	背景故事 (家乡、家庭、教育、关键事件)
Goals	list[Goal]	层级化目标 (优先级 + 截止日期 + 进度)
EmotionalState	struct	VAD 三维情绪模型 (愉悦度、唤醒度、支配感)

PersonaBuilder 接收自然语言描述,通过 LLM 生成完整人格 JSON。System Message 采用”冻结核心 + 动态上下文”双层结构:核心层 (身份、价值观、目标) 初始化后不变,上下文层 (处境、记忆、笔记) 每 tick 注入,确保人格一致性与环境感知的平衡。

3.2 世界引擎与关系双轨评估 (M2/M3)

世界引擎采用离散时间模拟 (Tick-Based),每个 tick 顺序执行:构建上下文、注入 Agent、执行决策、后处理 (Tool 副作用、目标更新、关系演化)、推送 SSE 事件。

Agent 之间的关系不是简单的“好友/敌人”二分,而是一个 [-1,+1] 的连续分数。分数的更新采用两条路径协同工作:

- 快速路径 (每 tick):检测对话中的关键词 (如“谢谢”“讨厌”“一起”),匹配 7 种交互类型,即时更新关系分数。1ms 内完成,保证每 tick 都有反馈。
- 深度路径 (每 3 tick,异步):让 LLM 阅读全部事件,评估关系变化。能捕捉关键词无法识别的微妙情感 (如反讽、暗示),但需要数秒。

快速路径保证实时性,深度路径补充准确性,两者协同使关系演化既实时又准确。

7 种交互类型及其对关系分数的影响:

类型	Δ	典型关键词
友好 (friendly)	+0.10	谢谢、喜欢、欣赏、支持
敌对 (hostile)	−0.15	讨厌、滚、闭嘴、嘲讽
合作 (cooperative)	+0.08	一起、合作、讨论、分享
竞争 (competitive)	−0.05	比、赢、挑战、超过
支持 (supportive)	+0.12	我相信你、别担心、有我在
轻视 (dismissive)	−0.10	随便、无所谓、别烦我
好奇 (curious)	+0.05	为什么、你怎么看、然后呢

关系分数在 $[-1, +1]$ 区间内累积演化, $A \rightarrow B$ 和 $B \rightarrow A$ 独立计算, 确保同一对 Agent 可以存在不对称关系。快速路径保证每 tick 都有关系更新, 深度路径补充微妙的情感信号, 两者协同使关系演化既实时又准确。

3.3 叙事、竞技与干预 (M4–M8)

叙事引擎 (M5) 将 Agent 经历转化为 9 种文体叙事 (小说/日记/信件/播客等), 保持 Agent 口吻一致。**竞技引擎** (M4) 支持辩论、面试、路演、盲测和大乱斗五种模式, 裁判从四维度评分。**控制台** (M6) 提供上帝视角仪表盘和决策模式识别。**干预台** (M7) 支持运行时注入事件和私密指令。**档案馆** (M8) 实现实验回放、模板沉淀和报告导出。

4 团队协作引擎 (M9)

M9 实现完整的多 Agent 协作工作流。以下聚焦三个核心设计决策。

4.1 设计决策一：文件系统与消息通信

方案：Agent 间通过共享文件系统协作，而非消息队列或 API 调用。

理由：文件系统天然具备**可追溯性**（所有协作留痕）、**可调试性**（随时查看任意步骤的输入输出）和**可回放性**（实验可完整复现）。每步工作区自动复制上游成功步骤的产出文件给下游，确保信息传递不依赖 Agent 的”记忆”。

隔离规则：Team 工作区与 Worker 工作区位于不同父目录，互不干扰。Team 工作区包含共享任务描述 (TASK.md)、团队配置 (TEAM.json) 和每步骤的上下文文件。

4.2 设计决策二：LLM 分解 + 规则兜底

方案：任务分解优先使用 LLM，LLM 不可用时降级为基于角色的规则分配（8 种预设角色模板）。

理由：LLM 分解灵活但不可靠（可能返回不存在的 Agent 名字），规则分解可靠但死板。两者结合既保证了正常路径的智能性，又保证了降级路径的可用性。安全网机制自动将 LLM 返回的名字映射为合法 UUID。

角色演化：每个步骤成功后，LLM 作为”团队观察者”评估 Agent 实际表现并动态调整其角色描述，使团队分工在执行中自我优化。

4.3 设计决策三：团队诊断与容错

团队诊断：DiagnosticsEngine 实时监控参与度（静默过久触发告警）、参与度评分（基于发言占比）和冲突检测（同一 tick 内过多并行操作时标记）。

容错机制：步骤失败时，保留已完成上下文，仅对剩余步骤重新分解，避免从零开始。Worker 内部具备 JSON 解析重试和 LLM 调用指数退避机制。

5 评测基准 (M10)

M10 的设计核心是**可解释的行为分析**，即：不仅给分，还要解释”为什么”。

5.1 设计决策一：鲁棒性为什么是跨任务指标

问题：单次任务中的”鲁棒性”无法衡量、LLM 存在随机性，一个 Agent 在一次模拟中可能碰巧表现很好。

方案：单任务中的鲁棒性只是占位值。通过一个 Agent 在多次任务中得分是否稳定评估。用统计中的**变异系数**（Coefficient of Variation）来衡量。设 n 次评测的分数为 $s_1, s_2, \dots, s_n$ ，定义：

$$\mu = \frac{1}{n} \sum_{i=1}^n s_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (s_i - \mu)^2}, \quad cv = \frac{\sigma}{\mu}$$

则鲁棒性指标为：

$$R = \max(0, 100 - cv \times 100)$$

当所有任务分数完全一致时 $cv = 0$, $R = 100$ （满分）；分数波动越大， cv 越大， R 越低。

设计考量：失败任务的零分也进入聚合，避免只统计成功样本导致结果虚高。例如，某 Agent 在 3 次评测中得分为 80、82、78，则 $\mu = 80$, $\sigma \approx 1.63$, $cv \approx 0.02$, $R \approx 98$ ，则表现极其稳定。

5.2 设计决策二：行为指纹为什么用信息熵

问题：如何量化”Agent 的行为是否一致”？简单的统计量（如平均值）无法捕捉行为分布的特征。

方案：思路很直接：统计 Agent 使用了哪些工具、各用了多少次。如果总是用同一种工具，说明行为很聚焦；如果均匀地使用各种工具，说明行为很分散。用 Shannon 信息熵（Shannon, 1948）把这个直觉变成公式。设 Agent 在 T 个 tick 内共调用 N 次工具，各工具调用频率为 $n_1, n_2, \dots, n_k$ ($\sum n_i = N$)，定义概率分布 $p(x_i) = n_i/N$ ：

$$H(X) = - \sum_{i=1}^k p(x_i) \log_2 p(x_i)$$

最大熵 $H_{\max} = \log_2 k$ （均匀分布时取到）。定义**行为一致性分数**：

$$C = 1 - \frac{H(X)}{H_{\max}}$$

$C \in [0, 1]$ ： $C \rightarrow 1$ 表示行为高度聚焦（几乎只用一种工具）， $C \rightarrow 0$ 表示行为高度分散（均匀使用所有工具）。

具体示例：Agent A 在 10 次调用中全部使用同一工具（observe），则 $H = 0$, $C = 1.0$ ，行为高度聚焦，为”观察型”Agent。Agent B 均匀调用 3 种工具各 3-4 次，则

$H \approx 1.585 = H_{\max}$, $C \approx 0.0$, 行为高度分散, 为”多面手”Agent。信息熵有效区分了两种截然不同的行为模式。

5.3 设计决策二-b：决策风格如何量化分类

问题：一致性分数 C 只能衡量行为的聚焦/分散程度, 无法区分”聚焦于什么”。

方案：从 Agent 的动作关键词中提取四类决策风格并量化占比。设 Agent 的动作关键词集合为 A , 四类关键词集分别为：

- 分析型 K_α : 分析、判断、计划
- 直觉型 K_β : 想要、尝试、选择
- 果断型 K_γ : 决定、坚持、接受
- 谨慎型 K_δ : 避免、拒绝、妥协

各风格占比定义为 $\alpha = |\{a \in A : a \in K_\alpha\}|/|A|$, β 、 γ 、 δ 同理。当某一类占比 $> 30\%$ 时归为对应风格 (可叠加, 如”分析型 + 果断型”), 否则为”平衡型”。配合逐维度诊断证据链 (思考次数、消息长度、回复标准差等), 评测结果完全可解释。

5.4 设计决策三：诊断评分的设计动机

问题：传统评测只给分数。用户无法判断这个分数是否合理。

方案：诊断评分从原始事件证据中逐维度追溯评分原因。例如”思考次数偏少 (5 次), 内部推理不充分”或”回复长度标准差: 42, 能根据情境调整表达方式”。配合劣化趋势检测 (对比最近 3 次与前 3 次评测), 可以看到完整的证据链。

6 游戏化场景 (M11)

M11 的核心创新是 MBTI 人格到游戏行为的端到端映射, 通过构建随机引擎, 实现人格不仅影响”说什么”, 还影响”怎么走”和”如何感受”。

6.1 设计决策一：MBTI 到行为的映射

选择：MBTI 直接推导移动参数 (间隔、范围、空闲率), 而非通过 prompt 间接影响。

映射规则：E/I 维度决定活动频率（外向型移动间隔短），T/F 维度决定行为倾向，N/S 维度决定探索范围。4 种人格组合各有独立的移动配置。

混合策略：移动目标选择采用物品寻求 + 社交接近 + 随机漫游的混合策略，使 Agent 的行为既有目的性又有随机性。

6.2 设计决策二：情绪引擎的多重触发

选择：EmotionEngine 集成五重触发链，而非单一的情绪模型。

- **对话触发：**关键词规则匹配 8 种情绪，对话气泡弹出时即时分析。
- **环境触发：**场景氛围（6 种场景各有情绪偏向）+ 邻近物品情绪影响。
- **社交共鸣：**近距离同情绪 Agent 互相放大，异情绪互相中和。
- **随机事件：**场景事件池（6 种场景共 25+ 事件）定期触发。
- **衰减回归：**情绪强度自然向中性回归。

理由：单一触发（如只有对话触发）不够真实。真实的情绪受多种因素共同影响，例如对话内容、所处环境、社交对象、突发事件和自然消退。五重触发链使 Agent 的情绪变化更加自然和可观察。

场景氛围与物品情绪的具体设计：

场景	氛围情绪	物品	情绪影响
图书馆	中性	床/沙发	疲倦
宿舍	疲倦	钢琴/喷泉	愉悦
教室	焦虑	自动售货机	兴奋
艺术中心	愉悦	电脑	困惑
实验室	困惑	地球仪	惊讶
樱花大道	愉悦 (×2)	树/盆栽	愉悦/中性

系统还维护了 25+ 条场景随机事件（如”图书馆突然停电””实验数据突然对上了””花瓣纷纷飘落”等），每 30–70s 随机触发一条，影响全体或随机个体。情绪引擎通过三条独立定时器协同工作：衰减定时器（每 30s 情绪向中性回归一级）、环境定时器（每 30s 场景与物品情绪微调）、随机事件定时器（30–70s 间隔），使 Agent 的情绪变化具有真实的时间节律。

6.3 设计决策三：目标预订防碰撞协议

选择：借鉴分布式系统的资源预订协议 (Reserve-Move-Release 三步)，而非简单的碰撞检测。

理由：碰撞检测是”事后补救” (碰撞发生后再处理)，目标预订是”事前预防” (移动前先确认目标格可用)。这确保了即使在高并发场景下，Agent 的空间行为也是**确定性的**，保障同一格不会被分配给两个 Agent。

7 管道编排引擎 (M12)

M12 的核心是一个支持复杂控制流的 DAG 管道编排引擎。

Worker 是管道中的执行单元，每个 Worker 通过显式状态机管理任务生命周期，共 6 个正常状态：IDLE (等待任务) → PLANNING (制定执行计划) → DECIDING (决策下一步) → EXECUTING (执行工具调用) → REFLECTING (反思结果) → DONE (完成)。反思阶段可回到 DECIDING (继续)、PLANNING (修订计划) 或直接到 DONE。系统设有最大步数限制防止无限循环。错误处理采用三级策略：RETRYABLE (网络超时等，自动重试)、SKIPPABLE (文件缺失等，Agent 自主决策)、FATAL (API 连续失败等，立即终止)。特别地，REFLECTING → PLANNING 的”修订”路径使 Worker 能根据交付验收反馈自动调整计划并重新执行，形成”执行 → 验收 → 修订”的闭环迭代。

7.1 设计决策一：Loop/Branch 的设计动机

问题：简单 DAG 只能表达”一次性”执行。现实中很多 workflows 需要迭代 (”分析 → 评审 → 不合格则修改”) 和条件跳转 (”如果搜索到足够信息则跳过补充搜索”)。

方案：在标准 DAG 的 FLOW 边之外，增加 LOOP (回边循环) 和 BRANCH (条件跳转) 两种边类型。条件值从节点**真实产出文件**中读取，支持正则匹配、数值比较和关键词检测三种模式。

执行策略：拓扑排序后按层级执行，同层节点最多 3 路并行。Loop 触发时将回边路径上的节点重新入队，Branch 触发时 BFS 标记被绕过节点为跳过。

7.2 设计决策二：确定性硬校验的设计动机

问题：LLM 验收员本身也可能产生”幻觉”，它可能误判一个有算式错误的交付为”通过”。这是 LLM 系统的固有风险。

方案：在 LLM 验收之前，先让程序自己检查，而不是让 LLM 去“看”算式对不对（可能看错），不如直接让程序算一遍。具体有两道检查：

1. **算式 AST 求值：**对交付物中的数学表达式，解析为抽象语法树（AST）后直接求值，判断计算是否正确。例如“ $3 + 5 \times 2$ ”解析为 `Add(3, Mul(5, 2))`，求值得 13，与交付结果比对。
2. **工具台账核验：**检查 Agent 声称执行的工具调用是否在机器台账中有真实记录，查验工具名是否存在、参数是否合法、返回值是否已写入产出文件。

硬校验发现的问题**强制标记为失败**，即使 LLM 验收员误判也无法绕过。

设计考量：确定性规则（快速硬校验， $O(1)$ 判断）与 LLM 判断（慢速语义评估）互相保险。其核心洞察是：**用确定性对抗随机性**，规则校验不会“幻觉”，而 LLM 判断可能“幻觉”，两条独立路径防止同源错误。

7.3 设计决策三：自动修订机制

选择：首次验收不通过时，将问题反馈送回 Worker 自动修订，最多两轮复验。

理由：验收不视作终点，而是迭代的一部分。自动修订使 Agent 有机会纠正错误，而不是简单地标记失败。超过修订上限后，当前工作区保留，用户可手动补充。

8 技术亮点总结

1. **人格直接决定行为：**MBTI 不仅影响 Agent 的对话风格，还直接决定移动频率、社交倾向和情绪变化，从“说什么”到“怎么走”再到“如何感受”，人格参数贯穿了 Agent 的全部行为维度。
2. **“快慢结合”双路径互保：**关键词匹配能在 1ms 内给出初步判断，但不够准确；LLM 深度评估需要数秒，但更可靠。在交付验收中，程序直接算一遍算式对不对、查一遍工具调用记录是否真实存在，不依赖 LLM 判断。这一模式在关系评估、任务分解、交付验收中反复出现。
3. **用信息熵量化行为：**统计 Agent 的工具使用习惯，用信息熵公式计算行为一致性分数，将“人格是否影响行为”从主观判断变为可计算的数学命题。配合逐维度诊断证据链和决策风格分类，评测结果完全可解释。

9 前端交互层

前端采用 React 18 + TypeScript + Vite，核心技术选型：

- UI: TailwindCSS + Lucide Icons
- 状态管理: React Query（服务端状态）+ Zustand（客户端状态）
- 实时通信: useSSE hook 封装 EventSource，支持自动重连和事件类型分发
- 游戏渲染: Phaser 3 集成 M11 游戏化场景
- 图表: Recharts 绘制雷达图、折线图等

前端包含 20 个主要页面，按功能分组：

模块	页面	功能描述
通用	Home, GuidePage, Settings	首页、功能引导、设置
M1	AgentFoundry, AgentModule	人格铸造、Agent 管理
M2/M3	SoloTheater, GroupSandbox, GameScene	单人/群体场景、游戏渲染
M4	Arena, DuelArena	竞技场、对决
M5	NarrativeFactory	叙事生成
M6/M7	ControlPanel, DirectorIntervention	控制台、干预
M8	Archive, ShowcaseWall	档案、展示
M9	TeamDashboard	团队监控
M10	BenchLab	评测实验
M12	WorkerBench, PipelinePage, PipelineEditor	Worker、管线管理、图形化编辑器

其中 PipelineEditor 提供基于 React Flow 的图形化管线编辑器，支持拖拽创建节点、连线定义依赖、实时运行监控。GameScene 集成 Phaser 3 渲染 M11 游戏化场景，Agent 在瓦片地图上自主移动、社交和情绪表达。

10 数据库设计

表名	说明
AgentRow	Agent 持久化（人格 JSON、情绪状态、指纹 JSON）
WorldRow	世界配置（场景、Agent 列表、Tick 数）
TeamRow	团队配置（成员、状态）
PlanRow	团队执行计划（任务、步骤、进度、结果）
BenchRun	评测运行（配置、状态、得分）
BenchResult	单条评测结果（指标、事件 JSON）
MemoryRow	Agent 长期记忆

数据库访问通过 `async_session()` 管理，使用 `aiosqlite` 驱动实现全异步操作。

11 总结

Life Lab 通过七大阶段的持续迭代，构建了一个功能完备且具有技术深度的 AI Agent 社会实验平台。系统的核心竞争力在于：

1. **完整性**：12 大模块覆盖 Agent 全生命周期，从人格铸造到行为评测到实验沉淀。
2. **深度**：用信息论量化行为一致性、用程序级校验解决交付可信度、Loop/Branch 管道支持迭代控制流，为解决 Agent 系统中的真实痛点提供可能的方案。
3. **一致性**：人格参数从对话到游戏行为的全链路映射，”人格驱动行为”不是口号，而是可验证的工程事实。
4. **工程成熟度**：关键词检测保证实时性，LLM 深度评估保证准确性，程序级校验防止 LLM 判断出错。

未来工作方向：更精细的人格-行为映射模型（超越 MBTI 粗粒度分类，引入大五人格连续谱）、3D 场景支持（Three.js/WebGPU 沉浸式观察）、更大规模社会模拟（50+ Agent 复杂社会结构涌现）、以及基于强化学习的 Agent 行为优化。最终目标是构建一

个真正的”AI 社会实验室”——让研究者能够以前所未有的深度和规模，探索 AI Agent 的社会性行为。